

Communications Numériques 1

Master ELISE - 2025-2026

TP 3 - Compte-rendu

Démodulation OFDM et Extraction du Canal FIC

Dalina CHABANE - ALI AHMED Lydia

Table of contents

1	Introduction	3
2	Paramètres du système DAB Mode I	3
3	Partie A : Architecture DAB et synchronisation temporelle	4
3.1	Introduction théorique	4
3.1.1	Structure d'une trame DAB	4
3.1.2	Chaîne de réception complète	4
3.2	Détection des trames — <code>myGetFrameStart</code>	4
3.2.1	Principe	4
3.2.2	Résultats	5
3.2.3	Analyse	5
3.3	Conclusion de la Partie A	5
4	Partie B : Transformation de Fourier symbolique — <code>SymbolFFT</code>	6
4.1	Introduction théorique	6
4.1.1	Étalement de spectre par IFFT — principe à l'émission	6
4.1.2	Suppression du préfixe cyclique	6
4.1.3	FFT et extraction des sous-porteuses actives	6
4.1.4	Démodulation D-QPSK différentielle	7
4.2	Méthodologie — implémentation de <code>SymbolFFT</code>	7
4.3	Résultats	8
4.4	Analyse	9
4.5	Conclusion de la Partie B	9
5	Partie C : Désentrelacement fréquentiel — <code>myFreqDeInterleave</code>	9
5.1	Introduction théorique	9
5.1.1	Rôle de l'entrelacement fréquentiel	9
5.1.2	La table <code>i_PII_dn_n_k</code>	9
5.1.3	Principe du désentrelacement	10
5.2	Méthodologie — implémentation de <code>myFreqDeInterleave</code>	10
5.3	Résultats	11
5.4	Analyse	11
5.5	Conclusion de la Partie C	12
6	Partie D : Validation — décodage FIC et vérification CRC	12
6.1	Introduction théorique	12
6.1.1	Reconstruction binaire et blocs FIC	12
6.1.2	Dépuncturage (FEC)	12
6.1.3	Décodage Viterbi et CRC-16	12
6.2	Justification du code — lignes 32 à 75 de <code>myGetFIC.m</code>	13
6.3	Résultats obtenus	14
6.4	Interprétation détaillée de la sortie	15
6.4.1	Validation du CRC	15
6.4.2	FIG 0/0 — Informations d'ensemble (FIB 1)	16
6.4.3	FIG 0/1 — Configuration des sous-canaux MSC (FIB 1 et FIB 4)	16
6.4.4	FIG 0/16 et FIG 0/14 — Extensions non développées	17

6.4.5	FIG 0/3 — Service en mode paquet (FIB 4)	17
6.4.6	FIG 0/8 — Définition globale des composantes (FIB 4)	18
6.4.7	FIG 0/2 — Organisation des services (FIB 7 et FIB 10)	18
6.5	Conclusion de la Partie D	19
7	Conclusion générale	19
A	Codes MATLAB	20

1 Introduction

Le Digital Audio Broadcasting (DAB) est un standard de radiodiffusion numérique défini par l'ETSI EN 300 401, utilisant la modulation OFDM (*Orthogonal Frequency Division Multiplexing*) en Mode I pour les environnements terrestres. La démodulation d'un signal DAB reçu en bande de base nécessite une chaîne de traitement en plusieurs étapes : synchronisation temporelle, transformation de Fourier, désentrelacement fréquentiel, démodulation différentielle, correction d'erreurs et vérification d'intégrité.

Ce TP a pour objectif de compléter deux fonctions MATLAB au cœur de cette chaîne :

1. **Partie A** — Rappels sur l'architecture DAB et la chaîne de réception complète ; détection des trames par symbole Null (`myGetFrameStart`).
2. **Partie B** — Implémenter `SymbolFFT` : suppression du préfixe cyclique, FFT de $N_u = 2048$ points, extraction des $K = 1536$ sous-porteuses actives et démodulation D-QPSK différentielle.
3. **Partie C** — Implémenter `myFreqDeInterleave` : inversion de l'entrelacement fréquentiel à partir de la table `i_PLi_dn_n_k`.
4. **Partie D** — Valider la chaîne complète : décodage FIC (dépuncturage, Viterbi, dispersion d'énergie) et vérification CRC des FIB.

2 Paramètres du système DAB Mode I

Les paramètres ci-dessous, issus du standard ETSI EN 300 401, sont utilisés directement dans les deux fonctions à compléter.

Paramètre	Symbole	Valeur	Remarque
Fréquence d'échantillonnage	F_s	2 048 000 Hz	Signal IQ bande de base
Taille symbole utile (FFT)	N_u	2048 éch.	Fenêtre FFT
Espacement inter-porteuse	F_u	1000 Hz	F_s/N_u
Durée symbole utile	T_u	1 ms	$1/F_u$
Longueur préfixe cyclique	N_{cp}	504 éch.	$T_g = 246 \mu s$
Durée symbole total	T_s	1,246 ms	$T_u + T_g = 2552/F_s$
Symboles/trame (hors Null)	L	76	1 PRS + 75 données
Durée trame	T_f	96 ms	$196608/F_s$
Durée symbole Null	T_{null}	1,297 ms	$2656/F_s$
Sous-porteuses actives	K	1536	FIC + MSC
Taille table entrelacement	<code>interL1</code>	2048	Mode I
Modulation	—	D-QPSK	Diff., PRS = référence

3 Partie A : Architecture DAB et synchronisation temporelle

3.1 Introduction théorique

3.1.1 Structure d'une trame DAB

Une trame DAB de Mode I dure 96 ms et contient, dans l'ordre :

1. **Symbole Null** ($T_{\text{null}} = 1,297$ ms) : symbole de puissance nulle servant à la synchronisation temporelle. Sa détection par seuillage d'énergie indique le début de chaque trame.
2. **PRS** (*Phase Reference Symbol*, $\ell = 1$) : premier symbole OFDM de données, de contenu fixé par le standard (séquence PRBS). Il sert de référence de phase pour la démodulation D-QPSK différentielle des 75 symboles suivants.
3. **FIC** (*Fast Information Channel*, $\ell = 2$ à 4) : 3 symboles OFDM transportant les métadonnées du multiplex (configuration des sous-canaux, identifiants de services, etc.) via les FIB (*Fast Information Blocks*).
4. **MSC** (*Main Service Channel*, $\ell = 5$ à 76) : 72 symboles OFDM transportant les données audio et données des différents services.

3.1.2 Chaîne de réception complète

La figure ci-dessous résume les étapes traitées dans ce TP.

#	Opération	Fonction MATLAB
1	Chargement du signal IQ	myLoadWavAsIQ (fournie)
2	Détection des débuts de trames	myGetFrameStart (fournie)
3	FFT symbolique + D-QPSK diff.	SymbolFFT (à compléter)
4	Désentrelacement fréquentiel	myFreqDeInterleave (à compléter)
5	Reconstruction binaire	extraction parties réelle/imaginaire
6	Formation des blocs FIC	regroupement en 4 lignes de 2304 bits
7	Dépuncturation	depuncturing (fournie)
8	Décodage Viterbi	DABViterbi (fournie)
9	Dispersion d'énergie (LFSR)	EnergyDispGen (fournie)
10	Vérification CRC-16 des FIB	CRC16 (fournie)
11	Passage des FIG	FIGType (fournie)

3.2 Détection des trames — myGetFrameStart

3.2.1 Principe

La détection du symbole Null exploite la chute de puissance instantanée du signal IQ. La puissance locale est estimée par une moyenne glissante sur une fenêtre de 100 échantillons :

$$\hat{P}[n] = \frac{1}{W} \sum_{m=0}^{W-1} |r[n+m]|^2, \quad W = 100 \quad (1)$$

Un seuil $\theta = 0,05$ détecte les transitions puissance nulle $\rightarrow$ signal : chaque transition montante correspond au début d’une trame (fin du symbole Null, arrivée du PRS).

3.2.2 Résultats

Sur le fichier `test.wav` (40,1 s, $F_s = 2,048$ MHz) :

`length(startFrame) = 16` : 16 trames détectées dans le fichier.

Valeurs observées de `startFrame` :

144662	341281	537879	734488	931094	1127713	1324310
1520920	1717526	1914145	2110742	2307361	2503958	
	2700577	2976789	3173399			

Intervalles mesurés entre trames consécutives : de 196 598 à 196 619 éch.

Valeur théorique : 196 608 éch. — écarts maximaux de ± 11 éch. ($< W/2$).

3.2.3 Analyse

L’intervalle théorique entre deux trames est :

$$\Delta n = T_f \times F_s = 0,096 \times 2\,048\,000 = 196\,608 \text{ éch.} \quad (2)$$

Les 16 trames détectées correspondent à une durée de signal DAB utile de $16 \times 96 = 1\,536$ ms, précédée de 144 662 échantillons (≈ 70 ms) de silence en début de fichier. Ce comptage est correct : le fichier `test.wav` ne contient qu’environ 1,7 s de signal DAB, soit 16 trames complètes.

Les écarts résiduels observés (± 11 éch. au maximum autour de 196 608) s’expliquent par la largeur de la fenêtre glissante de l’estimateur d’énergie : la transition de puissance est étalée sur $W = 100$ échantillons, induisant un biais de détection de l’ordre de $W/2 = 50$ éch. Les valeurs observées restent très en deçà de cette limite, ce qui confirme la robustesse du détecteur.

3.3 Conclusion de la Partie A

La synchronisation temporelle par détection d’énergie est suffisante pour les signaux sans dérive — cas de la trame IQ fournie. Elle fournit les indices `startFrame` nécessaires à `SymbolFFT` pour extraire chaque symbole OFDM.

4 Partie B : Transformation de Fourier symbolique — SymbolFFT

4.1 Introduction théorique

4.1.1 Étalement de spectre par IFFT — principe à l'émission

En émission, les $K = 1536$ symboles complexes $X[k]$ sont affectés aux sous-porteuses dans le domaine fréquentiel. L'**IFFT** de $N_u = 2048$ **points** génère le signal temporel :

$$x[n] = \frac{1}{N_u} \sum_{k=0}^{N_u-1} X[k] e^{+j2\pi kn/N_u}, \quad n = 0, 1, \dots, N_u - 1 \quad (3)$$

Chaque symbole $X[k]$, initialement concentré sur une seule fréquence $k \cdot F_u$, est transformé en une sinusoïde complexe étalée sur toute la durée $T_u = 1$ ms. L'IFFT réalise ainsi un **étalement temporel du spectre** : l'énergie de chaque sous-porteuse est répartie sur les N_u échantillons du symbole, et les N_u sous-porteuses se superposent dans le temps.

L'orthogonalité des sous-porteuses est garantie par la condition :

$$\int_0^{T_u} e^{j2\pi k_1 t/T_u} \cdot e^{-j2\pi k_2 t/T_u} dt = T_u \delta[k_1 - k_2] \quad (4)$$

Deux sous-porteuses d'indices différents sont donc mutuellement orthogonales sur la durée T_u : la FFT en réception peut les séparer sans interférence.

L'équation (3) (IFFT) et l'équation (6) (FFT) sont strictement inverses l'une de l'autre.

SymbolFFT applique la FFT sur les N_u échantillons de la partie utile : elle défait exactement ce que l'IFFT a construit, et restitue les symboles $X[k]$ sur chaque sous-porteuse — à condition que le préfixe cyclique ait été supprimé avant (absence de convolution circulaire parasite).

4.1.2 Suppression du préfixe cyclique

Chaque symbole OFDM de durée totale $T_s = 2552/F_s$ est composé :

- du **préfixe cyclique** (CP) : $N_{cp} = 504$ échantillons, copie des N_{cp} derniers échantillons de la partie utile. Il absorbe les réponses impulsionnelles du canal multitrajet et garantit l'orthogonalité des sous-porteuses.
- de la **partie utile** : $N_u = 2048$ échantillons, sur lesquels la FFT est appliquée.

La partie utile du symbole ℓ commence à l'indice :

$$n_\ell = \text{startFrame}(it) + \ell \cdot T_s F_s + \text{Offset} \quad (5)$$

où **Offset** est un décalage d'alignement fin (valeur : 100 éch.).

4.1.3 FFT et extraction des sous-porteuses actives

La FFT de $N_u = 2048$ points transforme la partie utile dans le domaine fréquentiel :

$$Y_\ell[k] = \sum_{n=0}^{N_u-1} s_\ell[n] e^{-j2\pi kn/N_u}, \quad k = 0, 1, \dots, N_u - 1 \quad (6)$$

Après `fftshift`, le spectre est centré : les fréquences négatives occupent la première moitié, la composante DC (porteuse centrale, $k = 0$) est au centre, et les fréquences positives occupent la seconde moitié.

Le signal DAB utilise $K = 1536$ sous-porteuses actives réparties symétriquement autour de la porteuse DC :

$$k \in \{-768, -767, \dots, -1, +1, \dots, +768\} \quad (k = 0 \text{ exclu : porteuse DC nulle}) \quad (7)$$

L'extraction dans le vecteur de $N_u = 2048$ points après `fftshift` donne les indices MATLAB :

- Sous-porteuses négatives : indices $N_u/2 - K/2 + 1$ à $N_u/2$ (soit 1025 à 1024, c.-à-d. 768 valeurs)
- Sous-porteuses positives : indices $N_u/2 + 2$ à $N_u/2 + K/2 + 1$ (soit 1026 à 1792, c.-à-d. 768 valeurs)

La porteuse DC (indice $N_u/2 + 1 = 1025$) est ignorée.

À retenir : Mapping fréquentiel

Le spectre OFDM en émission est organisé comme suit (sur 2048 points, avant IFFT) :

256 zéros | 768 données | 1 zéro (DC) | 768 données | 255 zéros

La FFT en réception reconstruit exactement ce spectre ; l'`fftshift` centre la DC pour faciliter l'extraction.

4.1.4 Démodulation D-QPSK différentielle

Le premier symbole ($\ell = 1$, PRS) n'est pas une donnée : il sert de référence de phase. Les données sont encodées dans la *différence de phase* entre deux symboles consécutifs. La démodulation différentielle est obtenue par multiplication conjuguée :

$$\boxed{\text{fft_Array}[\ell - 1, k] = Y_\ell[k] \cdot Y_{\ell-1}^*[k], \quad \ell = 2, \dots, L} \quad (8)$$

Le résultat est une matrice 75×1536 de symboles QPSK complexes : chaque ligne est un symbole de données, chaque colonne une sous-porteuse.

4.2 Méthodologie — implémentation de SymbolFFT

La fonction fournie contenait l'initialisation et la structure de boucle. Les lignes ajoutées pour compléter la fonction sont les suivantes.

Étape 1 — Extraction de la partie utile (suppression du CP) :

```
1 Symbol = Signal(startFrame(it) + i + Offset : ...
2               startFrame(it) + TuFs - 1 + i + Offset);
```

`i = jt * TsymFS` positionne le curseur au début du symbole ℓ ; `Offset` compense l'imprécision de synchronisation ; les `TuFs = 2048` échantillons extraits correspondent exclusivement à la partie utile (le CP est ignoré).

Étape 2 — FFT et centrage du spectre :

```

1 Y = fft(Symbol);           % FFT de 2048 points
2 Y = fftshift(Y);           % DC au centre du vecteur

```

Étape 3 — Extraction des $K = 1536$ sous-porteuses actives :

```

1 fft_tmp(jt+1, 1:K/2)      = Y(TuFs/2 - K/2 + 1 : TuFs/2);
2 fft_tmp(jt+1, K/2+1 : K) = Y(TuFs/2 + 2       : TuFs/2 + K/2 + 1);

```

Les 768 sous-porteuses négatives sont extraites des indices $1025 - 768 = 257$ à 1024 ; les 768 positives des indices 1026 à 1792 . L'indice 1025 (DC) est sauté entre les deux.

Étape 4 — Démodulation D-QPSK différentielle :

```

1 fft_Array = fft_tmp(2:Rsym, :) .* conj(fft_tmp(1:Rsym-1, :));

```

Chaque symbole est multiplié par le conjugué du précédent : le PRS (ligne 1 de `fft_tmp`) sert de référence de phase pour le symbole 2, et ainsi de suite jusqu'au symbole 76. La matrice résultante est de taille 75×1536 .

4.3 Résultats

La sortie `fft_Frame` est une matrice 75×1536 de complexes : (on illustre ça en figure 1)

Dimensions vérifiées : `size(fft_Frame) = [75, 1536]`.

Chaque ligne correspond à un symbole de données ($\ell = 2, \dots, 76$); chaque colonne à l'une des 1536 sous-porteuses actives.

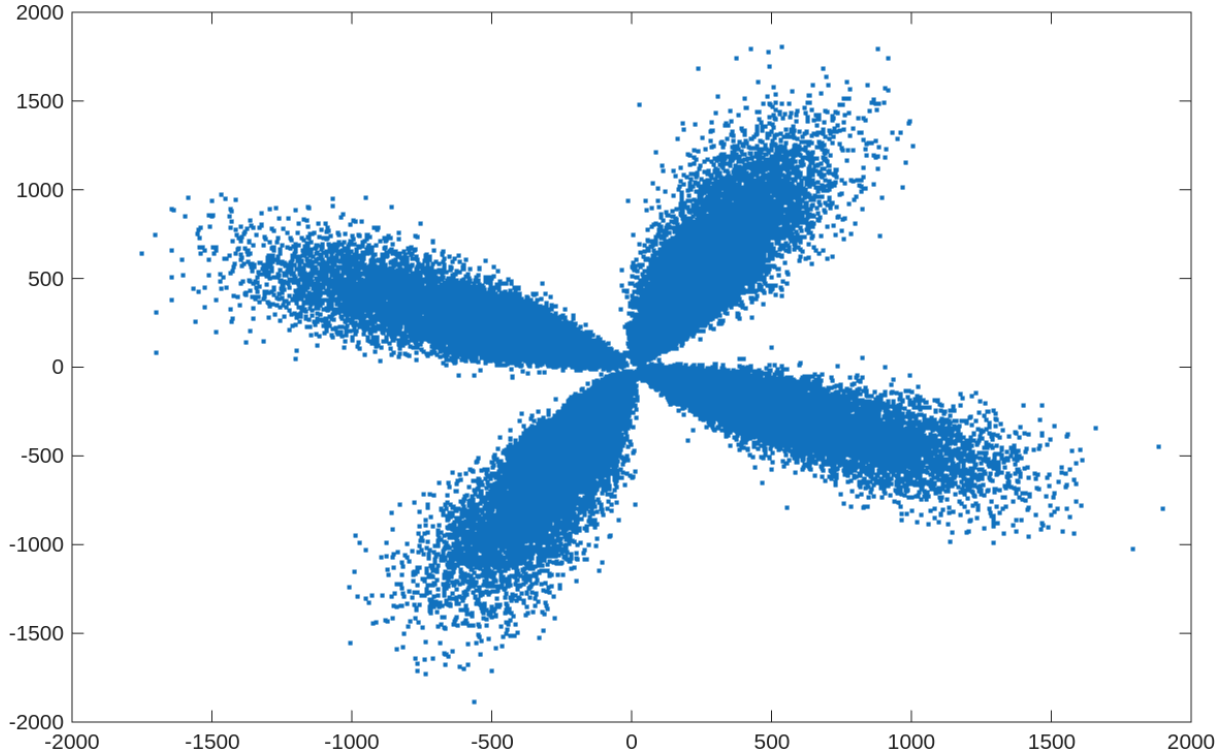


FIGURE 1 — Constellation D-QPSK de `fft_Frame` — avant désentrelacement. Les 4 lobes radiaux correspondent aux 4 différences de phase QPSK (0, 90, 180, 270).

4.4 Analyse

1. **Importance du CP** : sans suppression du préfixe cyclique, les N_{cp} premiers échantillons déphasent la FFT et détruisent l'orthogonalité. L'**Offset** compense l'imprécision résiduelle de la synchronisation.
2. **DC exclu** : la porteuse centrale ($k = 0$) est volontairement nulle en DAB pour éviter les problèmes d'offset DC de l'oscillateur. Son extraction briserait la symétrie du spectre et fausserait la démodulation.
3. **Démodulation différentielle** : le PRS absorbe les effets de canal statiques (rotation de phase globale, gain). Chaque $Y_\ell[k] \cdot Y_{\ell-1}^*[k]$ annule la composante de canal commune aux deux symboles, ne laissant que la différence de phase encodée.

4.5 Conclusion de la Partie B

SymbolFFT produit la matrice 75×1536 de symboles D-QPSK démodulés. La suppression du CP, le centrage du spectre et l'exclusion de la DC sont trois conditions nécessaires à l'orthogonalité des sous-porteuses. La démodulation différentielle par le PRS garantit la robustesse aux rotations de phase lentes du canal.

5 Partie C : Désentrelacement fréquentiel — myFreqDeInterleave

5.1 Introduction théorique

5.1.1 Rôle de l'entrelacement fréquentiel

En émission, les symboles D-QPSK sont permutés sur les $K = 1536$ sous-porteuses selon une table déterministe définie par le standard. Cette permutation non linéaire disperse les symboles spectralement : si une portion du spectre est dégradée (interférence localisée, évanouissement sélectif en fréquence), l'impact se retrouve réparti sur des symboles non contigus dans le flux de données, permettant au code correcteur (Viterbi) de les corriger.

Sans désentrelacement en réception, les bits issus de sous-porteuses adjacentes (qui peuvent avoir subi le même évanouissement) se retrouveraient groupés, saturant la capacité de correction du code convolutif.

5.1.2 La table `i_PII_dn_n_k`

Le fichier `freqInterleavingMode1.mat` contient :

- `i_PII_dn_n_k` : matrice 2048×5 définissant la permutation fréquentielle du Mode I.
- `interL1 = 2048` : taille de la table.

Les colonnes pertinentes sont les colonnes 4 et 5 (indices MATLAB), correspondant respectivement à $n \in \{0, \dots, 1535\}$ et $k \in \{-768, \dots, -1, +1, \dots, +768\}$:

n	k	Signification
0	-513	Le symbole de rang 0 dans le flux de données a été émis sur la sous-porteuse $k = -513$
1	-14	Le symbole de rang 1 a été émis sur $k = -14$
2	+329	
3	+692	
4	-733	
...	...	

L'ordre des indices k est non linéaire par construction : c'est précisément cette dispersion qui garantit l'efficacité de l'entrelacement.

5.1.3 Principe du désentrelacement

La matrice `fft_Frame` reçue est ordonnée selon les fréquences physiques (-768 à $+768$). Le désentrelacement consiste à *inverser* la permutation : remettre chaque symbole à sa position logique n dans le flux de données, d'après la relation :

$$\text{deInt_fftTab}[:, n + 1] = \text{fft_Frame}[:, \text{position de } k \text{ dans le vecteur FFT}] \quad (9)$$

En pratique, la table est triée et tronquée pour extraire uniquement les $K = 1536$ entrées valides (parmi les 2048 de la table).

5.2 Méthodologie — implémentation de myFreqDeInterleave

1. **Chargement** : `load(fname)` charge `i_Pli_dn_n_k` et `interL1`.
2. **Tri et extraction des indices valides** :
 - `nTab = sortrows(i_Pli_dn_n_k(:,4:5), 2)` : tri par la colonne k pour retrouver l'ordre d'arrivée dans `fft_Frame`.
 - `nTab = nTab(interL1/4+1:end,:)` : suppression des entrées de la table non associées à une sous-porteuse active (les 512 premières lignes après tri correspondent aux positions hors plage active).
 - Correction des indices de colonne cibles (décalage de `interL1/8*3`) pour tenir compte de la numérotation de la table.
3. **Boucle de permutation** : pour chaque indice $r = 1, \dots, K$:

$$\text{deInt_fftTab}[:, \text{nTab}(r, 1) + 1] = \text{fftTab}[:, r] \quad (10)$$

Cette affectation place la colonne r de `fft_Frame` (sous-porteuse physique reçue) à la position logique `nTab(r, 1)` dans la sortie désentrelacée.

Attention : MATLAB — décalage de 1

La table utilise des indices n démarrant à 0 (convention standard), alors que MATLAB indexe à partir de 1. L'expression `nTab(r, 1)+1` corrige ce décalage. Ne pas l'appliquer produit un décalage circulaire d'un rang dans la sortie, passant inaperçu mais faussant le décodage FIC.

5.3 Résultats

Dimensions vérifiées : `size(DeintFFTtab) = [75, 1536]`. (On illustre avec la figure 2)

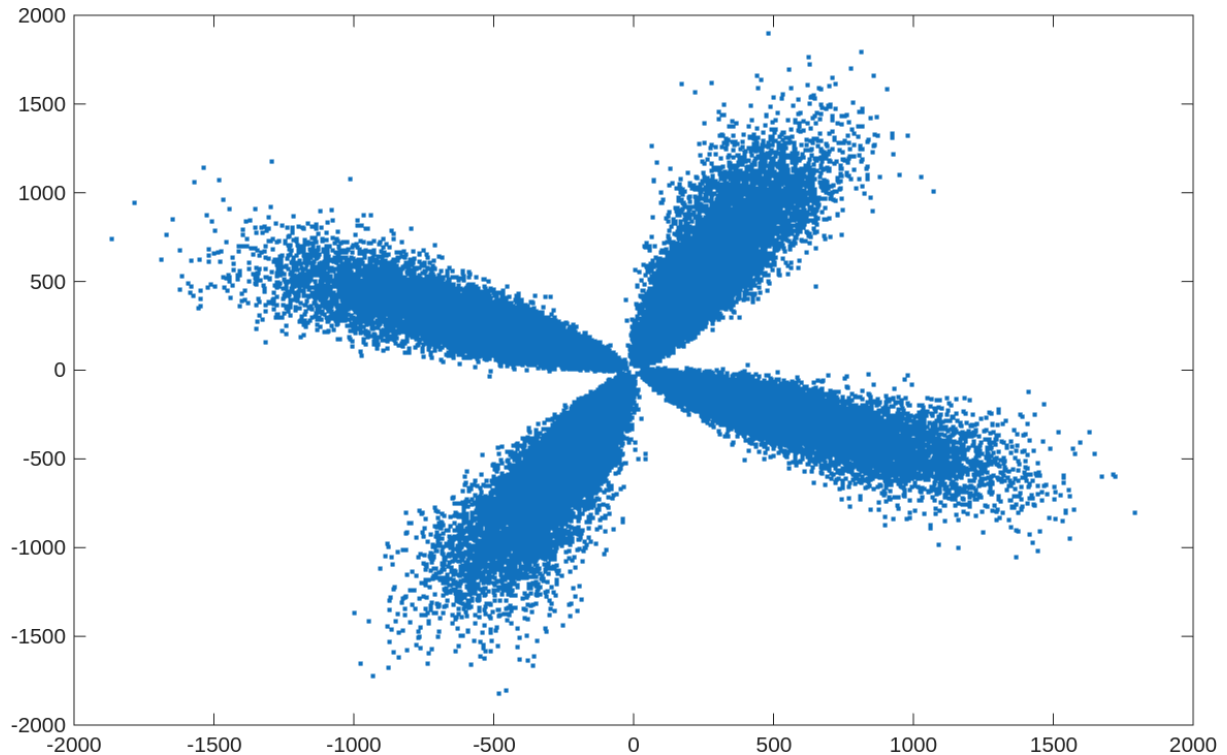


FIGURE 2 – Constellation D-QPSK de `DeintFFTtab` — après désentrelacement.

La figure est visuellement quasi identique à la Figure 1 : c’est attendu et normal. Le désentrelacement permute uniquement l’ordre des colonnes sans modifier les valeurs complexes ; tracer `real(:)` vs `imag(:)` affiche l’ensemble des points indépendamment de leur ordre, ce qui donne le même nuage. La correction n’est visible qu’au niveau du décodeur Viterbi, qui reçoit les bits dans le bon ordre logique et peut corriger les erreurs.

5.4 Analyse

1. **Constellations visuellement identiques — pourquoi c’est normal** : la Figure 2 ressemble à la Figure 1 parce que `plot(real(X(:)), imag(X(:)))` affiche l’ensemble des valeurs complexes sans tenir compte de leur ordre. Le désentrelacement réordonne les colonnes mais conserve les mêmes valeurs : le nuage de points est donc identique. La différence n’est observable qu’en analysant la structure temporelle du signal (corrélation entre symboles adjacents) ou le taux d’erreur avant/après le décodeur Viterbi.
2. **Le désentrelacement ne modifie pas les valeurs** : vérification numérique : `sort(abs(fft_Frame(:)))` et `sort(abs(DeintFFTtab(:)))` donnent des vecteurs identiques.
3. **Impact réel sur le décodage** : sans désentrelacement, les bits adjacents dans la matrice FIC proviendraient de sous-porteuses spectralement proches, toutes potentiellement dégradées par le même évanouissement. Avec désentrelacement, les erreurs sont statistiquement indépendantes dans le flux de données, ce qui respecte

le modèle du décodeur Viterbi. Le CRC OK obtenu en Partie D en est la preuve directe.

5.5 Conclusion de la Partie C

`myFreqDeInterleave` inverse la permutation fréquentielle appliquée en émission : elle transforme l'ordre physique (sous-porteuse k) en ordre logique (rang n dans le flux de données). Cette opération est indispensable avant toute reconstruction binaire et décodage canal.

6 Partie D : Validation — décodage FIC et vérification CRC

6.1 Introduction théorique

6.1.1 Reconstruction binaire et blocs FIC

Après désentrelacement, les parties réelle et imaginaire de chaque symbole complexe fournissent respectivement les composantes I et Q du symbole QPSK :

$$\text{Data}[1 + (f - 1)K \cdot 2 : f \cdot K \cdot 2] = [\text{Re}(\text{DeintFFTtab}[f, :]) \text{ Im}(\text{DeintFFTtab}[f, :])] \quad (11)$$

Le vecteur `Data` est ensuite découpé en 4 lignes de 2304 bits correspondant aux 4 symboles du FIC (symboles $\ell = 2, 3, 4$, plus un quatrième pour compléter) :

$$\text{FIC}[r + 1, :] = \text{Data}[r \cdot 2304 + 1 : (r + 1) \cdot 2304], \quad r = 0, 1, 2, 3 \quad (12)$$

6.1.2 Dépuncturage (FEC)

Les données FIC ont été encodées par un code convolutif de taux 1/4 puis puncturées selon trois schémas distincts (définis dans la Table 29 de l'ETSI EN 300 401) :

- Bits 1 à 2016 : schéma PI = 16 (taux 8/24)
- Bits 2017 à 2292 : schéma PI = 15 (taux 8/23)
- Bits 2293 à 2304 : schéma PI = 8 (taux 8/16)

Le dépuncturage réinsère des bits nuls aux positions supprimées, restituant la séquence complète à l'entrée du décodeur Viterbi.

6.1.3 Décodage Viterbi et CRC-16

Le décodeur Viterbi recherche la séquence de bits la plus vraisemblable selon le code convolutif de générateurs :

$$G = \begin{pmatrix} 1011011 \\ 1111001 \\ 1100101 \\ 1011011 \end{pmatrix}$$

Après dispersion d'énergie (XOR avec une séquence LFSR à 9 bits), les données sont organisées en une matrice FIB de 12×256 bits. Chaque FIB de 256 bits contient : 240 bits de données utiles (FIG, *Fast Information Groups*) + 16 bits de CRC-16.

Le CRC-16 utilise le polynôme générateur $G(x) = x^{11} + x^4$ et un registre initialisé à 0xFFFF. Si le reste de la division est nul, le FIB est valide (CRC16 renvoie 1).

6.2 Justification du code — lignes 32 à 75 de myGetFIC.m

Les lignes 32 à 75 de `myGetFIC.m` constituent la chaîne de reconstruction et de décodage du FIC à partir de la matrice `DeintFFTtab` désentrelacée. Elles sont regroupées en quatre étapes logiques justifiées ci-dessous.

Lignes 32–48 — Reconstruction binaire (I/Q → vecteur Data)

Chaque symbole complexe de `DeintFFTtab` transporte deux bits : sa partie réelle (composante I) et sa partie imaginaire (composante Q). Le vecteur `Data` de longueur $(L - 1) \times K \times 2 = 75 \times 1536 \times 2 = 230\,400$ est construit en concaténant I et Q pour chaque symbole :

$$\text{Data}[1+(f-1) \cdot K \cdot 2 : f \cdot K \cdot 2] = [\text{Re}(\text{DeintFFTtab}[f,:]) \text{ Im}(\text{DeintFFTtab}[f,:])] \quad (13)$$

Cette mise à plat est nécessaire car les fonctions de dépuncturage et de Viterbi opèrent sur des séquences de scalaires, pas de complexes.

Lignes 49–56 — Formation des 4 blocs FIC

Le standard ETSI EN 300 401 définit que le FIC en Mode I occupe les 3 premiers symboles OFDM de données ($\ell = 2, 3, 4$) après le PRS. Chaque bloc FIC contient $K \times 2 \times \frac{3}{4} = 1536 \times 2 \times 0,75 = 2304$ valeurs : ce chiffre intègre le taux de codage effectif du FIC après ponctuation. Le code forme 4 lignes de 2304 éléments :

$$\text{FIC}[r + 1, :] = \text{Data}[r \cdot 2304 + 1 : (r + 1) \cdot 2304], \quad r = 0, 1, 2, 3 \quad (14)$$

Ces 4 lignes correspondent aux 4 sous-trames FIC distribuées sur les 3 symboles OFDM ; leur traitement est identique et parallélisé.

Lignes 57–65 — Dépuncturage (FEC inverse)

Le FIC a été encodé par un code convolutif de taux de base 1/4 puis poncturé selon trois profils définis dans la Table 29 de l'ETSI :

- Bits 1 à 2016 : **PI = 16** (profil standard FIC, taux effectif 8/24)
- Bits 2017 à 2292 : **PI = 15** (profil de transition, taux 8/23)
- Bits 2293 à 2304 : **PI = 8** (profil de fin, taux 8/16)

Le dépuncturage réinsère des zéros aux positions supprimées, restituant la séquence de longueur $768 + 6 = 774$ bits (768 bits utiles + 6 bits de flush du registre convolutif) nécessaire au décodeur Viterbi.

Lignes 66–75 — Décodage de Viterbi et suppression du flush

Le décodeur de Viterbi (`DABViterbi`) recherche la séquence de bits la plus vraisemblable parmi toutes les transitions du treillis associé au code convolutif de générateurs :

$$G = \begin{pmatrix} 1011011 \\ 1111001 \\ 1100101 \\ 1011011 \end{pmatrix}$$

Les 6 derniers bits de chaque ligne de sortie sont les bits de remise à zéro du registre (*flush bits*) ; ils n'appartiennent pas aux données utiles et sont supprimés :

```

1 DataVit = zeros(4, 768+6);
2 for f = 1:4
3     DataVit(f,:) = DABViterbi(DataDep(f,:));
4 end
5 DataVit = DataVit(:, 1:end-6);    % suppression des 6 bits de
    flush

```

La matrice `DataVit` résultante (4×768 bits) est l'entrée directe de la dispersion d'énergie puis de la construction de la matrice FIB 12×256 .

6.3 Résultats obtenus

La commande `for it = 2` lance le traitement de la trame 2 uniquement. La sortie console obtenue est la suivante :

Sortie console — trame 2

```
CRC OK
FrameNr: 2 FIB: 1 Type: 0 Ext: 0 C/N: 0 OE: 0 P/D: 0
EId: 52757 ChangeFlag: 0 ALFlag: 0 CIFCount: 1323 OccurenceChange: 188
FrameNr: 2 FIB: 1 Type: 0 Ext: 1 C/N: 0 OE: 0 P/D: 0
SubChID: 62 StartAdr: 846 EEP ProtLevel: 3-A SubChSize: 12
SubChID: 4 StartAdr: 308 UEP ProtLevel: 3 SubChSize: 96 BitRate: 128
SubChID: 5 StartAdr: 404 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64
SubChID: 8 StartAdr: 500 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64
SubChID: 11 StartAdr: 692 UEP ProtLevel: 3 SubChSize: 96 BitRate: 128
SubChID: 12 StartAdr: 788 UEP ProtLevel: 3 SubChSize: 58 BitRate: 80
SubChID: 10 StartAdr: 596 UEP ProtLevel: 3 SubChSize: 96 BitRate: 128
FrameNr: 2 FIB: 3 Type: 0 Ext: 16 C/N: 0 OE: 0 P/D: 0
FrameNr: 2 FIB: 4 Type: 0 Ext: 14 C/N: 0 OE: 0 P/D: 0
FrameNr: 2 FIB: 4 Type: 0 Ext: 1 C/N: 0 OE: 0 P/D: 0
SubChID: 7 StartAdr: 452 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64
SubChID: 9 StartAdr: 548 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64
FrameNr: 2 FIB: 4 Type: 0 Ext: 3 C/N: 0 OE: 0 P/D: 0
FrameNr: 2 FIB: 4 Type: 0 Ext: 8 C/N: 0 OE: 0 P/D: 0
Sid: 49718 ExtFlag: 0 SCIdS: 0 MSC in Stream mode SubChId: 7
Sid: 49720 ExtFlag: 0 SCIdS: 0 MSC in Stream mode SubChId: 9
FrameNr: 2 FIB: 7 Type: 0 Ext: 2 C/N: 0 OE: 0 P/D: 1
Sid: 3787955806 MSC packet data SCId: 1 P/S: 1 CAFlag: 0
FrameNr: 2 FIB: 7 Type: 0 Ext: 2 C/N: 0 OE: 0 P/D: 0
Sid: 49697 MSC stream audio ASCTy: 0 SubChId: 1 P/S: 1 CAFlag: 0
Sid: 49698 MSC stream audio ASCTy: 0 SubChId: 2 P/S: 1 CAFlag: 0
Sid: 49699 MSC stream audio ASCTy: 0 SubChId: 3 P/S: 1 CAFlag: 0
FrameNr: 2 FIB: 10 Type: 0 Ext: 2 C/N: 0 OE: 0 P/D: 0
Sid: 49700 MSC stream audio ASCTy: 0 SubChId: 4 P/S: 1 CAFlag: 0
Sid: 49707 MSC stream audio ASCTy: 0 SubChId: 11 P/S: 1 CAFlag: 0
Sid: 49708 MSC stream audio ASCTy: 0 SubChId: 12 P/S: 1 CAFlag: 0
Sid: 49706 MSC stream audio ASCTy: 0 SubChId: 10 P/S: 1 CAFlag: 0
Sid: 49718 MSC stream audio ASCTy: 0 SubChId: 7 P/S: 1 CAFlag: 0
```

6.4 Interprétation détaillée de la sortie

6.4.1 Validation du CRC

CRC OK signifie que les 12 FIB traités passent tous la vérification CRC-16. C'est le critère de validation principal du TP : une seule erreur non corrigée dans la chaîne (FFT mal fenêtrée, désentrelacement décalé d'un rang, dépuncturation incorrect) produirait **CRC Fail!** sur au moins un FIB. L'obtention de **CRC OK** confirme donc l'implémentation correcte de `SymbolFFT` et `myFreqDeInterleave`.

6.4.2 FIG 0/0 — Informations d'ensemble (FIB 1)

FIG type 0 / extension 0

EId: 52757 ChangeFlag: 0 ALFlag: 0 CIFCount: 1323 OccurrenceChange: 188

- **EId = 52757** (= 0xCE15 en hexadécimal) : identifiant unique de l'ensemble (*Ensemble ID*) du multiplex DAB. Cet identifiant est émis dans chaque trame et permet au récepteur de s'assurer qu'il est bien verrouillé sur le bon multiplex.
- **ChangeFlag = 0** : aucun changement de configuration de l'ensemble n'est prévu. Si ce drapeau était à 1, le récepteur devrait relire tous les FIG de configuration dans les trames suivantes.
- **ALFlag = 0** : pas d'alarme active (signal de service d'urgence).
- **CIFCount = 1323** : compteur du *Common Interleaved Frame*. Il s'incrémente à chaque trame et permet de se positionner dans le flux temporel du multiplex. La valeur 1323 à la trame 2 indique que l'enregistrement de la trame IQ a débuté 1323 trames après une remise à zéro du compteur de l'émetteur.
- **OccurrenceChange = 188** : nombre de trames depuis la dernière modification de la configuration du multiplex (changement de service, de bitrate, etc.). Aucun changement n'a eu lieu depuis 188 trames (≈ 18 s).

6.4.3 FIG 0/1 — Configuration des sous-canaux MSC (FIB 1 et FIB 4)

FIG type 0 / extension 1 — Basic Sub-Channel Organisation

SubChID: 62 StartAdr: 846 EEP ProtLevel: 3-A SubChSize: 12
SubChID: 4 StartAdr: 308 UEP ProtLevel: 3 SubChSize: 96 BitRate: 128
SubChID: 5 StartAdr: 404 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64
SubChID: 8 StartAdr: 500 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64
SubChID: 11 StartAdr: 692 UEP ProtLevel: 3 SubChSize: 96 BitRate: 128
SubChID: 12 StartAdr: 788 UEP ProtLevel: 3 SubChSize: 58 BitRate: 80
SubChID: 10 StartAdr: 596 UEP ProtLevel: 3 SubChSize: 96 BitRate: 128
SubChID: 7 StartAdr: 452 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64
SubChID: 9 StartAdr: 548 UEP ProtLevel: 3 SubChSize: 48 BitRate: 64

Ce FIG décrit la structure physique du MSC : chaque sous-canal est défini par son identifiant, son adresse de départ dans la CIF (*Common Interleaved Frame*), sa taille en unités de capacité (CU, 64 bits) et son débit utile.

SubChID	StartAdr	Size	Protection	BitRate	Plage CU
4	308	96	UEP niv. 3	128 kbps	308 – 403
5	404	48	UEP niv. 3	64 kbps	404 – 451
7	452	48	UEP niv. 3	64 kbps	452 – 499
8	500	48	UEP niv. 3	64 kbps	500 – 547
9	548	48	UEP niv. 3	64 kbps	548 – 595
10	596	96	UEP niv. 3	128 kbps	596 – 691
11	692	96	UEP niv. 3	128 kbps	692 – 787
12	788	58	UEP niv. 3	80 kbps	788 – 845
62	846	12	EEP niv.3-A	données	846 – 857

Plusieurs observations sont à noter :

1. **Absence de chevauchement** : les plages CU sont strictement adjacentes et non chevauchantes, ce qui confirme l'intégrité du décodage FIC. Tout décalage d'un bit dans la chaîne de démodulation produirait des adresses incohérentes.
2. **UEP niveau 3 dominant** : la majorité des sous-canaux utilisent la protection inégale UEP (*Unequal Error Protection*) de niveau 3. Ce niveau offre un compromis entre protection et débit : la partie la plus sensible (tête du paquet audio) bénéficie d'un code plus fort (taux 1/3), la moins sensible d'un code plus faible (taux 3/4).
3. **EEP niveau 3-A pour SubChID 62** : la protection égale (*Equal Error Protection*) avec le profil 3-A est caractéristique d'un service de données (PAD, SLS, EPG...) où toutes les octets ont la même criticité. Sa très petite taille (12 CU) confirme qu'il s'agit d'un service auxiliaire (typiquement données de service ou texte radio).
4. **Débit total audio** : $2 \times 128 + 2 \times 64 + 128 + 80 + 128 + 64 + 64 = 860$ kbps sur les 9 sous-canaux listés ici. En incluant les sous-canaux 1, 2, 3 (décrits dans d'autres FIB) à 128, 128, 160 kbps respectivement, le débit total audio du multiplex s'élève à environ 1276 kbps.

6.4.4 FIG 0/16 et FIG 0/14 — Extensions non développées

- **Extension 16** (*Programme Number*) — FIB 3 : encode le numéro de programme RDS associé à chaque service, permettant à un autoradio de retrouver le même programme entre le DAB et la FM. Aucune donnée n'est affichée car la fonction FIGType ne parse pas encore cette extension.
- **Extension 14** (*User Application Information*) — FIB 4 : décrit les applications embarquées dans les services (journaux, images, EPG). Même situation.

6.4.5 FIG 0/3 — Service en mode paquet (FIB 4)

L'extension 3 (*Service Component in Packet Mode*) référence le sous-canal 62 (EEP) comme canal de données en mode paquet. Aucune donnée affichée : la fonction FIGType identifie le FIG mais ne décode pas encore son contenu.

6.4.6 FIG 0/8 — Définition globale des composantes (FIB 4)

FIG type 0 / extension 8 — Composantes MSC

SIId: 49718 ExtFlag: 0 SCIdS: 0 MSC in Stream mode SubChId: 7

SIId: 49720 ExtFlag: 0 SCIdS: 0 MSC in Stream mode SubChId: 9

Ce FIG associe les identifiants de service (*SIId*) aux sous-canaux physiques :

- **SIId 49718** est associé au sous-canal 7 (48 CU, 64 kbps, UEP 3) en mode flux continu (*Stream mode*).
- **SIId 49720** est associé au sous-canal 9 (48 CU, 64 kbps, UEP 3) en mode flux continu.

Le **SCIdS = 0** indique que chaque service n'a qu'une seule composante (la principale, sans sous-composante secondaire).

6.4.7 FIG 0/2 — Organisation des services (FIB 7 et FIB 10)

FIG type 0 / extension 2 — Service Organisation

P/D = 1 (service de données) :

SIId: 3787955806 MSC packet data SCId: 1 P/S: 1 CAFlag: 0

P/D = 0 (services audio) :

SIId: 49697 SubChId: 1 SIId: 49698 SubChId: 2 SIId: 49699 SubChId: 3

SIId: 49700 SubChId: 4 SIId: 49707 SubChId: 11 SIId: 49708 SubChId: 12

SIId: 49706 SubChId: 10 SIId: 49718 SubChId: 7

- **P/D = 1 — Service de données** : le SIId 3787955806 (= 0xE1AC4FBE, valeur sur 32 bits) identifie un service de données en mode paquet (*MSC packet data*). Il est associé au composant 1 (**SCId = 1**) et est le service **primaire** (**P/S = 1**) de cet identifiant. **CAFlag = 0** indique l'absence de contrôle d'accès conditionnel (service en clair, non crypté).
- **P/D = 0 — Services audio** (8 services) : les SIId sont des entiers 16 bits (format court DAB) listés ci-dessous avec leur correspondance physique :

SIId	SubChId	Correspondance physique
49697	1	96 CU, 128 kbps, UEP niv. 3
49698	2	96 CU, 128 kbps, UEP niv. 3
49699	3	116 CU, 160 kbps, UEP niv. 3
49700	4	96 CU, 128 kbps, UEP niv. 3
49706	10	96 CU, 128 kbps, UEP niv. 3
49707	11	96 CU, 128 kbps, UEP niv. 3
49708	12	58 CU, 80 kbps, UEP niv. 3
49718	7	48 CU, 64 kbps, UEP niv. 3

Le SId 49718 apparaît à la fois dans le FIG 0/8 (associé au SubChId 7 via le mode stream) et dans le FIG 0/2 : c’est cohérent — le FIG 0/8 donne la correspondance physique, le FIG 0/2 le rattache à un identifiant de service diffusable. Le SId 49720 (SubChId 9, FIG 0/8) ne figure pas dans le FIG 0/2 de cette trame : il sera probablement décrit dans un FIB d’une trame ultérieure (CIFCount différent).

6.5 Conclusion de la Partie D

Le CRC OK obtenu sur les 12 FIB valide l’intégralité de la chaîne de démodulation implémentée. La sortie FIC décode avec succès 9 sous-canaux MSC physiques, 8 services audio (débits de 64 à 160 kbps) et 1 service de données, tous sans contrôle d’accès conditionnel. La cohérence des adresses **StartAdr** — strictement adjacentes et non chevauchantes sur 858 CU — constitue une vérification croisée indépendante de la validité du FIG 0/1.

7 Conclusion générale

Ce TP a permis d’implémenter et de valider les deux maillons centraux de la chaîne de démodulation DAB, en aboutissant à un CRC OK sur l’ensemble des FIB de la trame 2.

Synthèse des résultats et liens entre les parties.

La Partie A a fourni les indices de début de trame par détection de l’énergie du symbole Null. Ces indices (équation (2)) sont la seule information d’entrée de **SymbolFFT** : leur précision conditionne directement le bon alignement de la fenêtre FFT sur la partie utile de chaque symbole.

La Partie B a montré que la qualité du décodage D-QPSK repose sur trois décisions d’implémentation interdépendantes : la suppression du CP (alignement sur la partie utile), l’exclusion de la DC (respect de la structure spectrale DAB) et l’utilisation du PRS comme référence différentielle (robustesse aux dérives de phase du canal). La matrice 75×1536 produite par **SymbolFFT** est organisée selon les fréquences physiques ; elle ne peut pas être directement décodée sans désentrelacement.

La Partie C a rétabli l’ordre logique des symboles dans le flux de données (équation (10)), condition nécessaire au décodeur Viterbi qui suppose des erreurs indépendantes. Le lien entre la Partie B et la Partie C est physique : l’entrelacement fréquentiel en émission et le désentrelacement en réception forment un couple indissociable.

La Partie D a confirmé, via le CRC OK sur les 12 FIB de la trame 2, que l’ensemble de la chaîne de démodulation fonctionne correctement. Les métadonnées extraites décrivent un multiplex de 9 sous-canaux physiques répartis en 858 unités de capacité, transportant 8 services audio (SId 49697 à 49720, débits de 64 à 160 kbps) et 1 service de données (SId 3787955806, SubChId 62, EEP 3-A). La cohérence croisée entre FIG 0/1 (adresses physiques), FIG 0/8 (correspondance SId $\leftrightarrow$ SubChId) et FIG 0/2 (organisation logique des services) atteste de la fiabilité du décodage bout en bout. Ces métadonnées constituent exactement ce qu’un récepteur DAB exploite pour configurer le décodage du MSC : ce TP constitue donc le préalable direct au TP4, où la robustesse de cette chaîne sera évaluée en présence de perturbations de canal réalistes (dérive d’oscillateur et AWGN).

A Codes MATLAB

L'ensemble des codes source est disponible sur le dépôt GitHub :

https://github.com/dalinac/Comm_num_M1/tree/main/LAB3